

Corso di Laurea triennale in Ingegneria e Scienze Informatiche

Image Denoising Technique with Neural Network

Relatore:

Prof. Lazzaro Damiana

Co/Contro Relatore

Dott. Ezio Greggio

Candidato:

Matteo Vanni

Matricola: 0000935584

Contents

1 Articolanzio

1.1 Panoramica sul problema

L'immagine denoising è il processo di rimozione di rumore da un'immagine.

Il rumore, che è causato da svariate fonti, quali foto fatte in condizioni di scarsa illuminazione o problemi che corrompono i file, causa perdita d'informazione sull'immagine.

Cos'è il rumore? Un'aggiunta casuale di pixel che non appartengono all'immagine originale e ce ne sono di varie tipologie:

Impulse Noise(IN) dove i pixel sono completamente diversi da quelli attorno. Esistono due categorie di IN: Salt and Pepper Noise(SPN) e Random Valued Impulse Noise(RVIN).

Additive White Gaussian Noise(AWGN) cambia ogni pixel dall'originale di una piccola quantità.

1.2 Utilizzo di modelli di Deep Learning

È essenziale rimuovere il rumore e ristabilire l'immagine originale dove riottenere l'immagine originale è importante per prestazioni robuste o ricostruire le informazioni mancanti è molto utile, come immagini astronomiche di oggetti molto lontani.

Le reti neurali convoluzionali lavorano bene con le immagini e ne utilizzeremo N, menzionate in alcuni paper di ricerca e compareremo i risultati di ogni modello.

1.3 Dataset utilizzati

Il primo dataset usato per addestrare i modelli sarà Oxford-IIIT Pet da Tensorflow, in modo poi da testare i modelli con immagini che non conoscono da altri dataset(colonscopie).

```
1 def load_dataset(split='train', img_size=(256,256), batch_size
    ↪ =16):
2     #Carica il dataset Oxford-IIIT Pet da tfds e applica il #
    ↪ preprocessing.
3
4     #Ritorna un dataset in batch.
5
6     # as_supervised=False per mantenere dict
7     dataset = tfds.load('oxford_iiit_pet', split=split,
    ↪ as_supervised=False)
```

```

8      # Preprocessing
9      dataset = dataset.map(lambda sample: preprocess(sample,
    ↪      img_size))
10     # Ottimizza il caricamento
11     dataset = dataset.shuffle(1024).batch(batch_size).prefetch(tf
    ↪      .data.AUTOTUNE)
12     return dataset

import numpy as np

def incmatrix(genl1,genl2):
    m = len(genl1)
    n = len(genl2)
    M = None #to become the incidence matrix
    VT = np.zeros((n*m,1), int) #dummy variable

    #compute the bitwise xor matrix
    M1 = bitxormatrix(genl1)
    M2 = np.triu(bitxormatrix(genl2),1)

    for i in range(m-1):
        for j in range(i+1, m):
            [r,c] = np.where(M2 == M1[i,j])
            for k in range(len(r)):
                VT[(i)*n + r[k]] = 1;
                VT[(i)*n + c[k]] = 1;
                VT[(j)*n + r[k]] = 1;
                VT[(j)*n + c[k]] = 1;

        if M is None:
M = np.copy (VT) else: M = np.concatenate ((M, VT), 1) VT = np.zeros ((n * m,1), int

```

Kvasir-seg: 1000(MILLEH) immagini di polipi,
 gli animali di arvard, le colonscopie

2 Decsrizione delle reti

2.1 Rumore

La prima funzione di rumore è u'aggiunta di rumore casuale a ogni pixel dell'immagine.

```

13 def add_noise(x, noise_factor=0.1):
14
15     x: immagine in input
16     noise_factor: fattore di rumore da aggiungere
17     Ritorna:
18         x_noisy: immagine con rumore aggiunto
19
20     noise = tf.random.normal(shape=tf.shape(x), mean=0.0, stddev
    ↪ =1.0)
21     x_noisy = x + noise_factor * noise
22     return x_noisy

```

E FAI RUMORE **QU** Prima funzione di rumore -i Random noise con fattore 0.3 massimo Rumore gaussiano Rumore a più layer

2.2 Autoencoder

Il primo approccio è stato quello di usare l'autencoder dell'articolo da cui sto copiando paro paro tutto.

Spiegazione del numero di layer usati e del tipo di mse loss fun, dam e learning rate di 1e-3

```

23 def build_autoencoder(input_shape):
24
25     Costruisce un autoencoder convoluzionale per immagini a
        ↳ colori.
26     Parametri:
27         input_shape: forma dell'immagine in input (es. (32,32,3))
28     Ritorna:
29         autoencoder: modello compilato
30
31     input_img = Input(shape=input_shape)
32
33     # Encoder
34     x = Conv2D(64, (3,3), activation='relu', padding='same')(
        ↳ input_img)
35     x = MaxPooling2D((2,2), padding='same')(x)
36     x = Conv2D(64, (3,3), activation='relu', padding='same')(x)
37     encoded = MaxPooling2D((2,2), padding='same')(x)
38
39     # Decoder
40     x = Conv2D(64, (3,3), activation='relu', padding='same')(
        ↳ encoded)
41     x = UpSampling2D((2,2))(x)
42     x = Conv2D(64, (3,3), activation='relu', padding='same')(x)
43     x = UpSampling2D((2,2))(x)
44     decoded = Conv2D(3, (3,3), activation='sigmoid', padding='
        ↳ same')(x)
45
46     autoencoder = Model(input_img, decoded)
47     autoencoder.compile(optimizer='adam', loss='
        ↳ binary_crossentropy', metrics=['accuracy'])
48
49     return autoencoder
50
51 autoencoder = build_autoencoder(input_shape=(img_size[0],
        ↳ img_size[1], 3))
52 autoencoder.summary()
```

2.3 RIDNet

stessa cosa dell'autencoder (vedere se aggiungere tutto il codice gigante del setup)

```

53 # MeanShift: sottrae (o aggiunge) la media RGB
54 @register_keras_serializable( MeanShift )
55 class MeanShift(Layer):
56     def __init__(self, rgb_mean, sign=-1, **kwargs):
57
58         rgb_mean: tupla con la media dei canali R, G, B.
59         sign: -1 per sottrarre la media, 1 per aggiungerla.
60
61         super(MeanShift, self).__init__(**kwargs)
62         self.rgb_mean = tf.constant(rgb_mean, dtype=tf.float32)
63         self.sign = sign
64
65     def call(self, x):
66
67         x      atteso in formato (batch, altezza, larghezza, 3)
68
69         return x + self.sign * self.rgb_mean
70
71 # BasicBlock: Conv2D seguita da attivazione ReLU
72 @register_keras_serializable( BasicBlock )
73 class BasicBlock(Layer):
74     def __init__(self, out_channels, kernel_size=3, stride=1,
75         ↪ use_bias=False, **kwargs):
76         super(BasicBlock, self).__init__(**kwargs)
77         self.conv = Conv2D(out_channels, kernel_size, strides=
78             ↪ stride,
79                             padding='same', use_bias=use_bias)
80         self.relu = ReLU()
81
82     def call(self, x):
83         return self.relu(self.conv(x))
84
85 # ResidualBlock: due convoluzioni con skip connection
86 @register_keras_serializable( ResidualBlock )
87 class ResidualBlock(Layer):
88     def __init__(self, out_channels, **kwargs):
89         super(ResidualBlock, self).__init__(**kwargs)
90         self.conv1 = Conv2D(out_channels, 3, strides=1, padding='
91             ↪ same')
92         self.relu = ReLU()
93         self.conv2 = Conv2D(out_channels, 3, strides=1, padding='
94             ↪ same')
95
96     def call(self, x):
97         residual = self.conv1(x)
98         residual = self.relu(residual)
99         residual = self.conv2(residual)

```

```

96         out = Add()(x, residual])
97         return ReLU()(out)
98
99 # EResidualBlock: versione estesa con convoluzioni a gruppi
100 @register_keras_serializable( EResidualBlock )
101 class EResidualBlock(Layer):
102     def __init__(self, out_channels, groups=1, **kwargs):
103         super(EResidualBlock, self).__init__(**kwargs)
104         self.conv1 = Conv2D(out_channels, 3, strides=1, padding='
        ↳ same', groups=groups)
105         self.relu = ReLU()
106         self.conv2 = Conv2D(out_channels, 3, strides=1, padding='
        ↳ same', groups=groups)
107         self.conv3 = Conv2D(out_channels, 1, strides=1, padding='
        ↳ valid')
108
109     def call(self, x):
110         residual = self.conv1(x)
111         residual = self.relu(residual)
112         residual = self.conv2(residual)
113         residual = self.relu(residual)
114         residual = self.conv3(residual)
115         out = Add()(x, residual])
116         return ReLU()(out)
117
118 # Merge_Run_dual: due rami convoluzionali con dilatazioni diverse
119 ↳ , poi merge e skip connection
120 @register_keras_serializable( MergeRunDual )
121 class MergeRunDual(Layer):
122     def __init__(self, out_channels, **kwargs):
123         super(MergeRunDual, self).__init__(**kwargs)
124         # Primo ramo
125         self.conv1a = Conv2D(out_channels, 3, strides=1, padding='
        ↳ same')
126         self.relu1a = ReLU()
127         self.conv1b = Conv2D(out_channels, 3, strides=1, padding='
        ↳ same', dilation_rate=2)
128         self.relu1b = ReLU()
129         # Secondo ramo
130         self.conv2a = Conv2D(out_channels, 3, strides=1, padding='
        ↳ same', dilation_rate=3)
131         self.relu2a = ReLU()
132         self.conv2b = Conv2D(out_channels, 3, strides=1, padding='
        ↳ same', dilation_rate=4)
133         self.relu2b = ReLU()
134         # Gogeta
135         self.conv3 = Conv2D(out_channels, 3, strides=1, padding='
        ↳ same')
136         self.relu3 = ReLU()

```



```

137     def call(self, x):
138         branch1 = self.relu1a(self.conv1a(x))
139         branch1 = self.relu1b(self.conv1b(branch1))
140
141         branch2 = self.relu2a(self.conv2a(x))
142         branch2 = self.relu2b(self.conv2b(branch2))
143
144         merged = Concatenate()([branch1, branch2])
145         merged = self.relu3(self.conv3(merged))
146         return Add()([merged, x])
147
148 # CAlayer: Channel Attention Layer
149 @register_keras_serializable( CAlayer )
150 class CAlayer(Layer):
151     def __init__(self, channel, reduction=16, **kwargs):
152         super(CAlayer, self).__init__(**kwargs)
153         self.channel = channel
154         self.reduction = reduction
155         self.conv1 = Conv2D(channel // reduction, 1, strides=1,
156             ↳ padding='same')
157         self.relu = ReLU()
158         self.conv2 = Conv2D(channel, 1, strides=1, padding='same'
159             ↳ , activation='sigmoid') #sigmoid: risultato tra 0 e
160             ↳ 1
161
162     def call(self, x):
163         # Calcolo della media globale per canale
164         y = tf.reduce_mean(x, axis=[1, 2], keepdims=True)
165         y = self.relu(self.conv1(y))
166         y = self.conv2(y)
167         return Multiply()([x, y])
168
169 # Block: combinazione di MergeRunDual, ResidualBlock,
170     ↳ EResidualBlock e CAlayer
171 @register_keras_serializable( Block )
172 class Block(Layer):
173     def __init__(self, out_channels, **kwargs):
174         super(Block, self).__init__(**kwargs)
175         self.merge_run_dual = MergeRunDual(out_channels)
176         self.residual_block = ResidualBlock(out_channels)
177         self.e_residual_block = EResidualBlock(out_channels)
178         self.ca = CAlayer(out_channels)
179
180     def call(self, x):
181         r1 = self.merge_run_dual(x)
182         r2 = self.residual_block(r1)
183         r3 = self.e_residual_block(r2)
184         out = self.ca(r3)
185         return out

```

```

183 # Creazione del modello
184 @register_keras_serializable( RIDNET )
185 def RIDNET(n_feats=64, rgb_range=1.0):
186
187     n_feats: numero di feature channels usate all'interno del
188         ↳ modello.<br>
189     rgb_range: scala dei valori RGB (ad es. 1.0 se l'input
190         ↳ normalizzato [0,1]).
191
192     rgb_mean = (0.4488, 0.4371, 0.4040)
193
194     input_layer = Input(shape=(None, None, 3))
195
196     # Sottosottrai la media (MeanShift con sign=-1)
197     sub_mean = MeanShift(rgb_mean, sign=-1)
198     x = sub_mean(input_layer)
199
200     # Testa: BasicBlock (conv + ReLU)
201     head = Conv2D(n_feats, 3, strides=1, padding='same',
202         ↳ activation='relu')(x)
203
204     # Serie di blocchi
205     b1 = Block(n_feats)(head)
206     b2 = Block(n_feats)(b1)
207     b3 = Block(n_feats)(b2)
208     b4 = Block(n_feats)(b3)
209
210     # Coda: convoluzione finale per ottenere 3 canali
211     tail = Conv2D(3, 3, strides=1, padding='same')(b4)
212
213     # Aggiungi la media (MeanShift con sign=+1)
214     add_mean = MeanShift(rgb_mean, sign=1)
215     res = add_mean(tail)
216
217     # Connessione residua a livello di immagine: somma con l'
218         ↳ input originale
219     output = Add()([res, input_layer])
220
221     model = Model(inputs=input_layer, outputs=output)
222     return model
223
224 model = RIDNET(n_feats=32, rgb_range=1.0)
225 model.compile(optimizer='adam', loss='mse', metrics=['accuracy'])
226     ↳ )
227 model.summary()

```

3 Analisi ed ottimizzazione

3.1 Prestazioni

PSNR è il metodo più comune per misurare la qualità delle immagini.

Il PSNR è definito come il rapporto tra il massimo valore possibile di un segnale e il valore del rumore che disturba la qualità della sua rappresentazione.

Normalmente misurato in una scala logaritmica in decibel(dB).

Data l'immagine originale(g) e l'immagine rumorosa(f), il PSNR è definito come:

$$PSNR = 20 \log_{10} \left(\frac{MAX_f}{\sqrt{MSE}} \right)$$

dove MAX_f è il valore massimo del pixel dell'immagine e si calcola come

$$MSE = \frac{1}{mn} \sum_0^{m-1} \sum_0^{n-1} \|f(i, j) - g(i, j)\|^2$$

mentre MSE (*Mean Square Error*) è l'errore quadratico medio tra l'immagine originale e quella rumorosa.

3.2 Analisi dei modelli

3.2.1 Autoencoder

La rete neurale è un autoencoder convoluzionale simmetrico progettato per la compressione e successiva ricostruzione di immagini RGB.

Nella fase di encoding, l'immagine in input viene inizialmente processata da un primo strato convoluzionale (Conv2D) con 64 filtri, ciascuno di dimensione 3×3 , seguito da un'operazione di sottocampionamento (MaxPooling2D) che riduce la risoluzione spaziale da 112×112 a 56×56 mantenendo invariato il numero di canali. Un secondo strato convoluzionale, anch'esso composto da 64 filtri, approfondisce l'estrazione delle caratteristiche locali, cui segue un ulteriore livello di pooling che porta la risoluzione a 28×28 pixel. A questo punto, l'informazione visiva è compressa in una rappresentazione latente di dimensione (28, 28, 64), che mantiene un buon bilanciamento tra capacità rappresentativa e riduzione della dimensionalità.

La fase di decoding ha l'obiettivo di ricostruire l'immagine originale a partire dalla rappresentazione compressa. In essa, un ulteriore strato convoluzionale affina la mappa delle caratteristiche, seguito da un'operazione di upsampling (UpSampling2D) che ripristina la dimensione spaziale a 56×56 .

Successivamente, un altro strato convoluzionale migliora la qualità della ricostruzione a tale scala. Un secondo upsampling riporta l'immagine alla dimensione originale di 112×112 pixel. Infine, un ultimo strato convoluzionale con 3 filtri produce l'output finale, riconvertendo il volume delle feature maps in un'immagine RGB.

L'architettura è completamente simmetrica rispetto alla fase di encoding e decoding e non prevede strati densi (fully connected), risultando pertanto particolarmente adatta alla gestione di immagini di dimensione variabile. Inoltre, la scelta di mantenere una rappresentazione latente relativamente "larga" ($28 \times 28 \times 64$) rispetto ad autoencoder più tradizionali consente una migliore preservazione dei dettagli visivi, rendendo questo modello particolarmente adatto a compiti di denoising o ricostruzione di immagini degradate.

Layer	Output Shape	Params #
Input layer	(None, 112, 112, 3)	0
Conv2D	(None, 112, 112, 64)	1,792
MaxPooling2D	(None, 56, 56, 64)	0
Conv2D 1	(None, 56, 56, 64)	36,928
MaxPooling2D 1	(None, 28, 28, 64)	0
Conv2D 2	(None, 28, 28, 64)	36,928
UpSampling2D	(None, 56, 56, 64)	0
Conv2D 3	(None, 56, 56, 64)	36,928
UpSampling2D 1	(None, 112, 112, 64)	0
Conv2D 4	(None, 112, 112, 3)	1,731
Architettura dell'autoencoder Model: functional		
Total params:	114,307	446.51 KB
Trainable params:	114,307	446.51 KB
Non-trainable params:	0	0.00 B

3.2.2 RIDNet

La rete RIDNet è composta da vari strati che interagiscono in modo complesso, e come si vede dalla tabella ogni passaggio è concatenato in modo sequenziale, dove l'output di un blocco è l'input del successivo ??.

Il flusso di dati inizia con l'Input Layer 5, il quale riceve un'immagine di dimensioni sconosciute (None, None, None, 3). Questo strato non ha parametri addizionali, essendo semplicemente un livello di ingresso.

Subito dopo, il Mean Shift 10 applica una trasformazione ai dati, mantenendo invariata la dimensione dell'immagine. Questo strato ha il compito di centralizzare l'intensità dei pixel.

Il primo blocco convoluzionale della rete è rappresentato dal Conv2D 250, che riduce progressivamente la dimensione spaziale dell'immagine mentre aumenta il numero di filtri da 3 a 32.

Successivamente, il flusso prosegue attraverso una serie di Block che non solo applicano operazioni convoluzionali ma anche attivazioni non lineari, garantendo una maggiore capacità di rappresentazione della rete. Ogni blocco (da Block 20 a Block 23) mantiene una struttura simile e continua ad applicare operazioni convoluzionali mantenendo gli stessi parametri.

L'output dell'ultimo blocco convoluzionale, il Conv2D 299, produce un'immagine di output della stessa dimensione dell'immagine di ingresso, con qualche parametro in meno e nuovamente 3 canali, questa è una prima previsione dell'immagine finale.

Infine, un ulteriore strato di Mean Shift 11 viene applicato per eseguire un altro aggiustamento del valore medio dei pixel nell'immagine di output. Anche in questo caso, non vengono introdotti nuovi parametri.

L'output finale della rete è generato dallo strato Add 413, che esegue una somma tra l'output di Mean Shift 11 e il risultato precedente. Questo step finale non introduce parametri addizionali, ma è cruciale per combinare le informazioni passate attraverso la rete, migliorando la qualità del risultato finale.

In sintesi, la RIDNet si basa su una sequenza di operazioni convoluzionali seguite da trasformazioni di "mean shift" e operazioni di somma, ottimizzando la qualità dell'immagine di output attraverso una struttura profonda e iterativa. La rete è progettata per ridurre progressivamente l'immagine, applicare convoluzioni e migliorare la qualità attraverso un processo di addizione che incorpora informazioni provenienti da differenti livelli della rete.

Layer (type)	Output Shape	Param #	Connected to
input_layer_5	(None, None, None, 3)	0	-
mean_shift_10	(None, None, None, 3)	0	input_layer_5[0][0]
conv2d_250 (Conv2D)	(None, None, None, 32)	896	mean_shift_10[0][0]
block_20 (Block)	(None, None, None, 32)	93,666	conv2d_250[0][0]
block_21 (Block)	(None, None, None, 32)	93,666	block_20[0][0]
block_22 (Block)	(None, None, None, 32)	93,666	block_21[0][0]
block_23 (Block)	(None, None, None, 32)	93,666	block_22[0][0]
conv2d_299 (Conv2D)	(None, None, None, 3)	867	block_23[0][0]
mean_shift_11	(None, None, None, 3)	0	conv2d_299[0][0]
add_413 (Add)	(None, None, None, 3)	0	mean_shift_11[0][0]
Architettura della RIDNet			
Params Model: functional_5			
Total params:	376,427	1.44 MB	
Trainable params:	376,427	1.44 MB	
Non-trainable params:	0	0.00 B	

3.3 Prima run

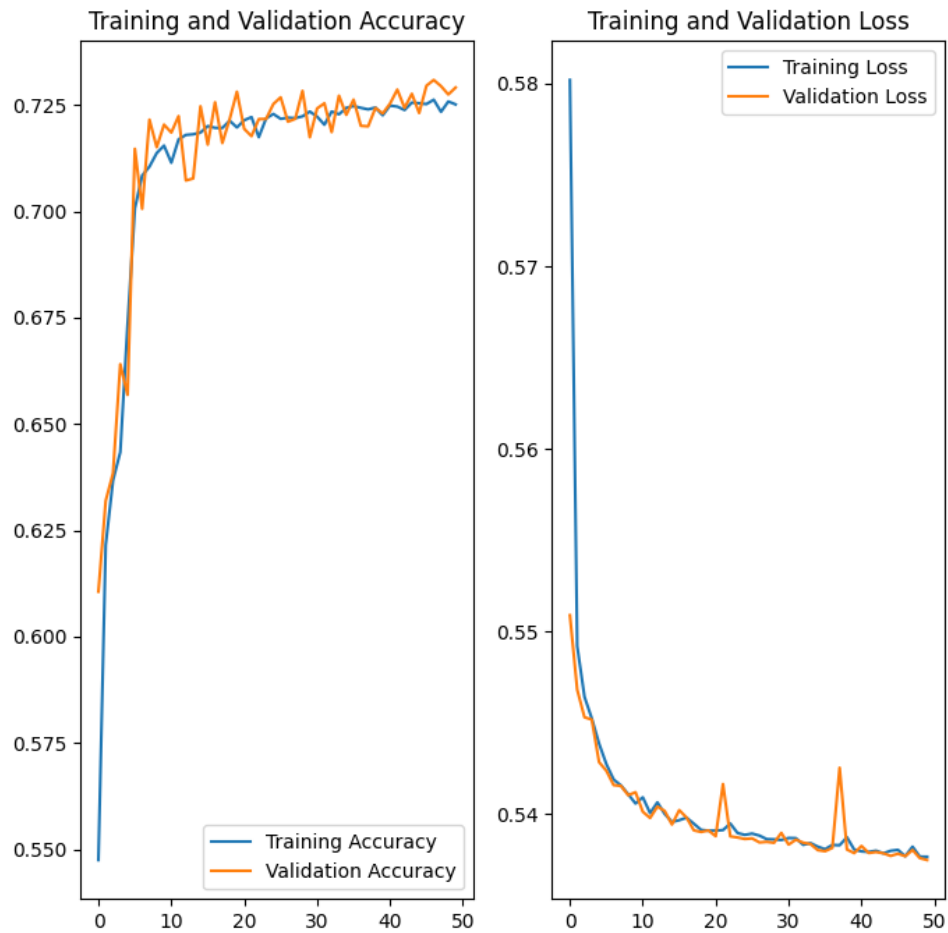
Il primo allenamento è stato sottoposto a entrambi i modelli con immagini RGB dal dataset tensorflow "oxford_iiit_pet" con risoluzione 112x112 pixel, un noise_factor di 0.4 e 50 epoche di training ed entrambe con una batch_size di 64.

Il dataset non presenta una suddivisione tra train, validation e test ma solo tra train e test, allora è stato suddiviso il training in 80% training e 20% validation.

3.3.1 Autoencoder

Una volta allenato il modello il risultato del training presenta un leggero overfitting sul validation set, mentre sempre nel validation sono presenti un paio di picchi nel loss (come da grafico ??

Figure 3.1: First training of the autoencoder



Poi è stata eseguita una predict e confronto del loss level tra immagini ricreate e reale con diversi livelli di rumore e vedere come va

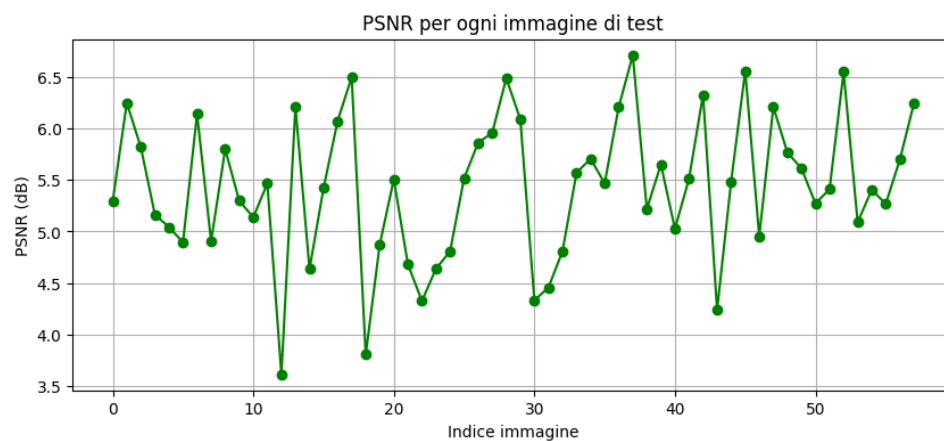


Figure 3.2: NON SO SE VA BENE COSÌ

la media PSNR sul test set è di 5.43 dB. Come si può notare dal grafico??

3.3.2 RIDNet

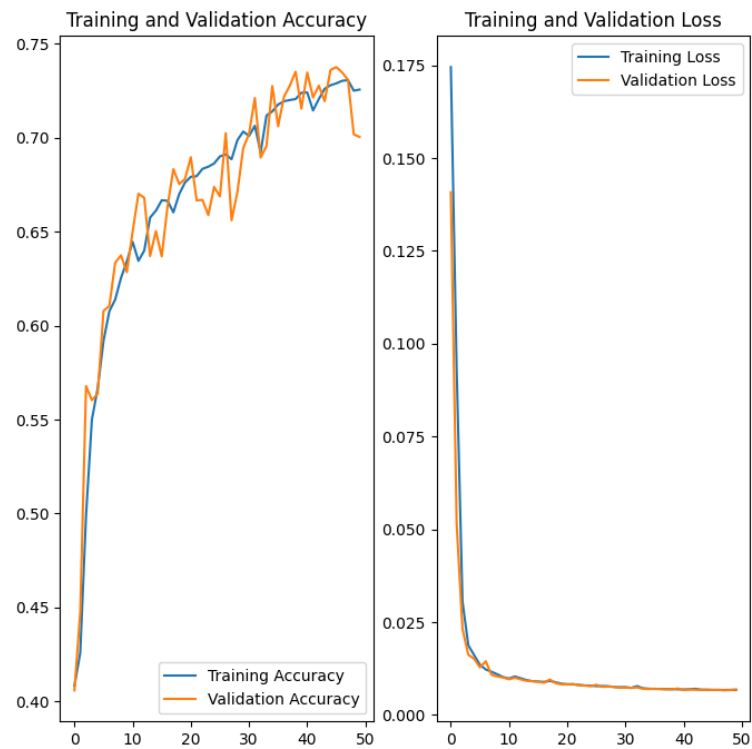


Figure 3.3: First training of the autoencoder

3.3.3 RIDNet

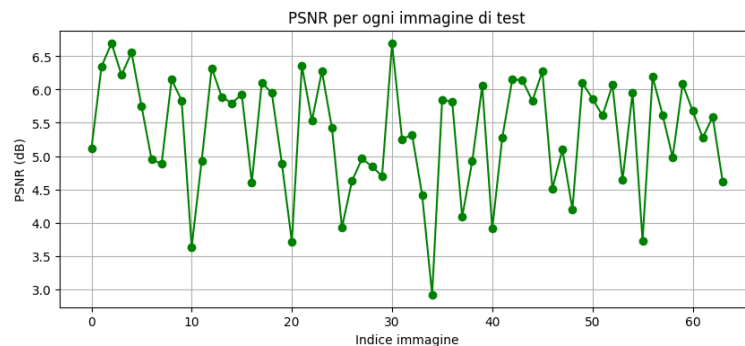


Figure 3.4: First psnr of the autoencoder

la media PSNR sul test set è di 5.37 dB. Come si può notare dal grafico

3.4 Seconda run

batch_size più piccola(?) o diverse dimensioni tra split del dataset e train; 50(?) epoche

3.5 Terza run

<QUI SAREBBE DA DOVER CAMBIARE LEARNING RATE E LOSS LEVEL>

3.6 Quarta run

<CAMBIO DELLE ARCHITETTURE DELLE DUE RETI PER MIGLIORARLA A LIVELLO DI PRESTAZIONI>

3.7 Quantizzazione dei modelli

3.7.1 Performance su dataset sconosciuti

4 Bibliografia

- nome $_i$